GenSpark

Using random and time, and Reading Error Messages

You can now check input, make decisions, and repeat. This article adds two tools from Python's standard library, `random` and `time`, plus a skill every programmer needs: reading the error messages Python gives you. Together they cover the rest of what the Number Guessing Game needs. `random` picks the secret number, `time` runs the Time Challenge, and error messages tell you what to fix when something breaks.

Modules and import

A module is a file of ready-made code that you can borrow. Python ships with many modules, called the standard library, and they need no installation. Third-party packages, such as `requests`, are different: you install them with `pip` inside your virtual environment.

You borrow a module with `import`, and you use it by writing the module name, a dot, and the tool you want:

python

```
import random

number = random.randint(1, 10)
print(number)
```

Put imports at the very top of your file. That makes it easy to see what your program depends on.

You may also see this style, which pulls out one tool by name:

python

```
from random import randint

number = randint(1, 10)
```

Both work. In this program, prefer `import random`. Writing `random.randint` tells the reader where the tool comes from.

Two import traps

- **Do not name your file after a module.** If your file is called `random.py`, Python imports your file instead of the real module, and you get strange errors. Use names like `guess.py` and `practice.py`.
- **Do not name a variable after a module.** If you write `time = 5` after `import time`, the name `time` no longer refers to the module, and `time.time()` fails.

The random Module

`random` produces unpredictable values. Strictly speaking they are "pseudo-random", meaning calculated by a formula that looks random. That is more than good enough for a game.

Tool	What it does	Example
`random.randint(a, b)`	A random whole number from a to b, **including both ends**	`random.randint(1, 6)`
`random.choice(items)`	One random item from a list	`random.choice(["red", "green", "blue"])`
`random.random()`	A random decimal from 0 up to, but not including, 1	`random.random()`

python

```
import random

die = random.randint(1, 6)
color = random.choice(["red", "green", "blue"])
```

```
print(f"You rolled {die} and drew {color}.")
```

Compare `randint` with `range()` from the loops article. `range(1, 6)` stops before 6, but `random.randint(1, 6)` can return 6. For a secret number from 1 to 100, write `random.randint(1, 100)`.

Making random numbers repeatable

A random program is hard to test, because you never know what number it picked. `random.seed()` fixes the starting point of the formula, so the same seed produces the same sequence every time:

python

```
import random

random.seed(7)
print(random.randint(1, 100))
random.seed(7)
print(random.randint(1, 100))    # the same number as the line above
```

Seeding is a testing tool. Do not leave it in a finished game, or the secret number would never change. You will see this idea again in Week 2, when you write automated tests.

Forcing a Value for Testing

There is a second way to make a random program testable: let yourself override the random value with a known one. The Number Guessing Game does this with a setting called `DEBUG_SECRET`. To write it, you need one new idea.

`None` is Python's value for "nothing here". It is not zero, and it is not an empty string. It means no value has been set. You check for it with `is` and `is not`:

python

```
import random

DEBUG_SECRET = None          # no forced number
# DEBUG_SECRET = 42          # uncomment this line to force the number 42
```

```
low = 1
high = 100

if DEBUG_SECRET is not None:
    secret_number = DEBUG_SECRET
else:
    secret_number = random.randint(low, high)
```

While testing, you set `DEBUG_SECRET = 42`. The secret number is now always 42, and you know the right answer. Before you commit your finished game, set it back to `None`. Use `is None` and `is not None` for this check, not `==`.

The time Module

`time.time()` returns the current time as a number of seconds. The number is large, because on a Mac it counts from January 1, 1970, but the starting point does not matter. What matters is that you can subtract two readings to find out how much time passed between them.

python

```
import time

start_time = time.time()
input("Press Enter when you are ready to stop. ")
elapsed = time.time() - start_time

print(f"That took {elapsed} seconds.")
```

The result is a float, such as `3.482917`. Three tools make it nicer to show:

python

```
print(f"{elapsed:.1f} seconds")     # :.1f shows one decimal place, e.g. 3.5
print(int(elapsed))                 # int() drops the decimals, so 3.9 becomes
print(round(elapsed))               # round() goes to the nearest whole numbe
```

The `:.1f` goes inside the braces of an f-string. Change the number for more or fewer decimals, such as `:.2f`. Note that `round()` sends an exact half to the nearest even number, so `round(2.5)` is 2 and `round(3.5)` is 4. That rarely matters for timers.

Pausing. `time.sleep(seconds)` pauses your program. It is useful for testing, such as waiting out a timer:

python

```
print("Wait for it...")
time.sleep(2)
print("Done")
```

`time` or `datetime`? The `time` module is for measuring how long something took. The `datetime` module, which you used in the Git lab with `date.today()`, is for calendar dates. Use `time` for timers and `datetime` for dates.

A note on accuracy. `time.time()` follows the computer's clock, and that clock can be adjusted, for example when your Mac syncs with a time server. Python also offers `time.monotonic()`, which only ever moves forward and is the safer choice for measuring durations in production code. You use it the same way: save `start = time.monotonic()`, then calculate `time.monotonic() - start`. This course uses `time.time()` because it is the most common, and either works for the projects.

Building a Countdown Timer

A countdown timer in a program has three parts:

1. **Save the start time once**, before the loop.
2. **Calculate the time that has passed** by subtracting the start from the current time.
3. **Compare** that to the limit.

python

```
import time

TIME_LIMIT = 30                    # seconds allowed
```

```
start_time = time.time()          # save the start ONCE, before the loop

attempts_used = 0
while attempts_used < 3:
    elapsed = time.time() - start_time
    remaining = max(0, TIME_LIMIT - elapsed)
    print(f"Time remaining: {round(remaining)} seconds")

    word = input("Say the magic word: ").strip().lower()
    attempts_used += 1

    # Check the clock right after the player answers
    if time.time() - start_time > TIME_LIMIT:
        print("Time is up!")
        break

    if word == "please":
        print("Correct!")
        break
```

Four details matter here:

- If you saved the start time inside the loop, the timer would restart on every guess. The player would never run out of time.
- `max(0, ...)` keeps the display from showing a negative number. `max()` returns the larger of its inputs.
- The display uses `round()`. With `int()`, a 30 second timer would show 29 on the first pass, because `int()` drops the decimals of 29.99.
- The timeout check comes before the answer check. As you saw in the decisions article, the order decides which condition wins. A correct answer that arrives after the deadline should lose.

input() cannot be interrupted

While `input()` waits, your whole program is paused. Python cannot update a countdown on screen or stop the player in the middle of typing. The clock keeps running in the background, though. So the program can only notice a timeout **after** the player presses Enter, by comparing the clock against the

limit. That is how the Number Guessing Game works. A live ticking countdown needs techniques beyond this week.

Testing a timer without waiting

You do not have to wait 60 seconds to test a 60 second timer. Make the limit a named setting at the top of the file, and change it while you test:

python

```
TIME_LIMIT = 5      # use 5 while testing, then change back to 60
```

Run the program, wait more than 5 seconds before you answer, and check that the timeout message appears. Remember to restore the real value before you commit.

Reading Error Messages

Sooner or later your program will stop with a wall of red text. Beginners often feel that means they failed. It does not. An error message is Python telling you exactly what went wrong and where. Learning to read it is one of the most useful skills you can build.

Anatomy of a traceback

Run this code and type `abc` when it asks:

python

```
guess_text = input("Your guess: ")
guess = int(guess_text)
print(guess + 1)
```

Python prints:

```
Traceback (most recent call last):
  File "/Users/alex/guess/guess.py", line 2, in <module>
    guess = int(guess_text)
            ^^^^^^^^^^^^^^^
```

```
ValueError: invalid literal for int() with base 10: 'abc'
```

Read it **from the bottom up**:

1. **The last line** names the type of error (`ValueError`) and says what happened. Read this first.
2. **The `File` line above it** gives the file and the line number (`line 2`). Go there.
3. **The code line below it** shows the exact line that failed. The `^^^^` markers, which Python 3.11 and later print, point at the part of the line that caused the problem.

When your code uses functions, tracebacks list several `File` lines, one for each step. Read the **last `File`** line first, because it is closest to the problem. In PyCharm, file paths in error output are usually clickable and jump you to the line.

The errors you will meet first

Error	What it means	Typical cause	Typical fix
`SyntaxError`	Python cannot understand the line	Missing colon, missing closing parenthesis or quote, `=` instead of `==`	Fix the spelling or punctuation on the line shown, or the line just above it
`IndentationError`	Spacing is wrong	A block that is not indented, or lines that do not line up	Indent the block with 4 spaces and keep each block consistent
`NameError`	A name does not exist	Misspelled variable, or used before it was set	Fix the spelling, or set the variable earlier
`TypeError`	A value has the wrong type for the operation	Adding text and a number, or comparing text to a number	Convert with `int()` or `str()`
`ValueError`	The type is right but the value is not	`int("abc")`	Check with `isdigit()` first
`ModuleNotFoundError`	Python cannot find the module	Misspelled `import` , or a package not installed	Fix the name, or `pip install` it in your virtual environment

`AttributeError`	The value has no such tool	Misspelled method, such as `random.randit`	Fix the spelling
`ZeroDivisionError`	Divided by zero	`10 / 0`	Check the divisor before dividing
`KeyboardInterrupt`	You pressed `Ctrl+C`	You stopped a running program	Nothing to fix. This is how you stop an infinite loop.

You will meet `IndexError` and `KeyError` in the weeks on lists and dictionaries.

Examples to recognize on sight

These are real messages from Python 3.12.

Missing colon:

```
  File "guess.py", line 2
    if guess < 5
               ^
SyntaxError: expected ':'
```

Missing indentation:

```
  File "guess.py", line 3
    print("x")
    ^
IndentationError: expected an indented block after 'if' statement on line 2
```

Misspelled variable:

```
  File "guess.py", line 2, in <module>
    print(gues)
          ^^^^
NameError: name 'gues' is not defined. Did you mean: 'guess'?
```

Misspelled tool:

```
AttributeError: module 'random' has no attribute 'randit'. Did you mean: 'ranc
```

Adding text and a number:

```
TypeError: can only concatenate str (not "int") to str
```

Comparing text to a number. This is the most likely error in your guessing game:

```
TypeError: '<' not supported between instances of 'str' and 'int'
```

It means you compared a string, such as the text from `input()`, with a number. Convert the text with `int()` before you compare.

Unclosed parenthesis:

```
  File "guess.py", line 1
    print("Hello"
          ^
SyntaxError: '(' was never closed
```

Two more tips about SyntaxError

Python reports where it noticed the problem, and that is sometimes one line after the real mistake. An unclosed parenthesis on line 5 may show up as an error on line 6. If the line Python points at looks fine, check the line above it.

Also, Python reports only the **first** syntax error it finds. When you fix it and run again, another one may appear. That is normal. Fix one thing at a time and rerun.

A routine for any error

1. **Do not panic, and do not change code yet.** Read the last line of the message.
2. **Say it in your own words.** "It says my text and number can't be added."

3. **Go to the line** Python names, and look at the line above it too.
4. **Change one thing**, then run again.
5. **If the message changes, that is progress.** You solved one problem and found the next.

When you ask a teammate or an AI assistant for help, paste the full error message and the code around it, not a description. The exact words are what make the problem easy to find.

Bugs Without Error Messages

The hardest bugs produce no error at all. The program runs, and the result is simply wrong: the attempt counter goes up on bad input, the timer restarts every guess, or a loop ends one pass too early. These are called logic errors, and Python cannot catch them because the code is valid.

The best tool for finding them is **print debugging**: print the values of your variables at key moments and compare them to what you expected.

python

```
while attempts_used < max_attempts:
    guess_text = input("Guess: ").strip()
    print(f"DEBUG: guess_text={guess_text!r}, attempts_used={attempts_used}")
```

The `!r` inside the braces prints the value with its quotes, so you can see blank entries and stray spaces clearly. Watch what the variables do when you type something unexpected, and find the first moment where they differ from what you predicted. Remove your debug lines before you commit.

Another useful habit is to trace the code by hand, as you did with loops. Write down each variable after every pass and compare with the real output.

PyCharm also has a built-in debugger that lets you pause a program and inspect its variables. To try it, click in the gray margin next to a line number to set a breakpoint, then click the bug icon to run in debug mode. Print debugging is enough for this week.

Putting It Together

This short program uses both modules: it picks random numbers, times the answer, and checks the clock after the player responds.

python

```python
import random
import time

TIME_LIMIT = 10          # seconds allowed to answer

first = random.randint(1, 12)
second = random.randint(1, 12)

start_time = time.time()
answer_text = input(f"What is {first} x {second}? ").strip()
elapsed = time.time() - start_time

if elapsed > TIME_LIMIT:
    print(f"Too slow! You took {elapsed:.1f} seconds.")
elif not answer_text.isdigit():
    print("That is not a whole number.")
elif int(answer_text) == first * second:
    print(f"Correct, in {elapsed:.1f} seconds!")
else:
    print(f"Wrong. The answer was {first * second}.")
```

Run it several times. Try answering correctly, answering wrong, typing letters, and waiting longer than 10 seconds. Notice that the timeout check is first, so a correct but late answer still loses. Then change `TIME_LIMIT` to 3 and see how it plays.

Common Mistakes

Mistake	What happens	Fix
Forgetting `import random`	`NameError: name 'random' is not defined`	Add the import at the top
Naming your file `random.py` or `time.py`	Strange errors on import	Rename your file
Expecting `random.randint(1, 100)` to exclude 100	Surprise at getting 100	`randint` includes both ends

Using `time` as a variable name	`time.time()` stops working	Pick another name, such as `start_time`
Saving the start time inside the loop	The timer restarts every pass	Save it once, before the loop
Checking the answer before the clock	A late correct answer wins	Check the timeout first
Showing remaining time with `int()`	A 30 second timer starts at 29	Use `round()`
Leaving `DEBUG_SECRET` or `TIME_LIMIT = 5` in your final code	The game is broken for real players	Restore `None` and `60` before you commit
Writing `if DEBUG_SECRET == None`	Works, but not the Python style	Use `is None` and `is not None`
Comparing the text from `input()` with a number	`TypeError`	Convert with `int()` first
Changing three things at once after an error	You cannot tell what fixed it	Change one thing and rerun
Ignoring the last line of the traceback	You miss the answer	Read from the bottom up

Try It Yourself

Create a file called `tools.py` and run it with `python3 tools.py`. Try these:

1. Roll a die: print a random number from 1 to 6.
2. Make a list of three colors, pick one at random with `random.choice`, and ask the user to guess it. Print whether they were right.
3. Call `random.seed(7)` and print `random.randint(1, 100)` twice, setting the seed again before the second one. Confirm both numbers match. Then remove the seed and run it again to see random numbers.
4. Measure how long the user takes to type their name. Print the time with one decimal place.
5. Count down from 5 to 1 with a `for` loop, using `time.sleep(1)` between numbers, then print

"Liftoff".

6. Add a `DEBUG_SECRET` setting to a program that picks a number from 1 to 20. Test that setting it to `7` always produces 7, and that `None` gives a random number.
7. Change the "Putting It Together" program to ask three math questions in a loop, with a 10 second limit for each one. Keep score and print it at the end.
8. Read the errors: run each line below in a file, one at a time, and write a plain sentence about what each message says.
    - `print("Hello"`
    - `print(score)`, with no `score` variable set
    - `print("5" + 3)`
    - `int("abc")`
    - `import randon`
    - `print(10 / 0)`
9. Fix this program. It has four bugs. Python will only show you one at a time, so fix and rerun until it works:

python

```
import random

secret_number = random.randint(1, 10)
guess_text = input("Guess 1 to 10: ")
if guess_text.isdigit()
    guess = int(guess_text)
    if guess = secret_number:
        print("Correct")
    else
        print("Wrong, it was " + secret_number)
```

Why This Matters for an FDE

On a client engagement you will often work in code you did not write, with systems you did not design, under time pressure. What distinguishes the engineer who is stuck from the one who is productive is rarely knowledge of the language. It is the ability to read an error message calmly, locate the cause, change one thing, and verify the result. Timers, retries, and timeouts also show up constantly in real systems, and so does the need to make unpredictable behavior repeatable enough to test.

Comprehension Check

1. What is the difference between `range(1, 6)` and `random.randint(1, 6)`?
2. Why must you save `start_time` before the loop and not inside it?
3. In a timed game, why does the program check the clock after `input()` returns and not while the player is typing?
4. What does `DEBUG_SECRET = None` mean, and what happens in the code when you change it to `42`? Why must you restore it before your final commit?
5. You see this error. In which order do you read it, and what is the problem?

```
Traceback (most recent call last):
  File "/Users/alex/guess/guess.py", line 9, in <module>
    if guess < secret_number:
       ^^^^^^^^^^^^^^^^^^^^
TypeError: '<' not supported between instances of 'str' and 'int'
```

6. A program runs without any error, but the attempt counter goes up when the player types letters. What kind of bug is this, and how would you find it?
7. Why is it a bad idea to name your own file `random.py`?
8. A 30 second timer shows `29 seconds` on its first pass when you format it with `int(remaining)`. Why, and how do you fix it?

← Previous: Loops: while, for, break, and continue

Next: Manual Testing and Writing a Test Plan →

0 comments so far

6

Add a comment...

Comment cannot be empty

Comment

No comments yet. Be the first to start the discussion!